

Software Project 3 Specification: Stock Data and Candlestick Pattern Viewer

Windows Forms Application for Stock Data Analysis

Project Overview

Project 3 will be an upgrade of project 2. In project 3, you will use your newly acquired knowledge of inheritance and polymorphism to improve your application. You will now allow the user to select and display candlestick patterns on the chart.

To do so, you will add a ComboBox control that will hold the names of all the patterns your application can recognize (e.g. Marubozu, Engulfing Pattern, etc....)

To that effect, you will provide an enhanced class called `aSmartCandlestick` which is to be derived from your `aCandlestick` class defined earlier in project 1. As we did in class, make sure that the `aSmartCandlestick` class has all the appropriate as well as all the necessary constructors (default, copy, etc....)

You will also need a method in your form to create a list of smart candlesticks from a list of regular candlesticks.

It is that list of smart candlesticks that you will process to find the location, therein, of candlestick patterns (e.g. Marubozu, Engulfing Pattern, etc....)

A use case is as follows. After selecting one or more stocks to display, your application is displaying the selected stocks in the charts associated with display forms. In each form, there will be the aforementioned ComboBox to select the pattern of interest. When the user clicks on a pattern in, an event will be fired by the ComboBox. In the event handler you can find out which pattern was selected. Your application should then scan your list of smart candlesticks and locate all the one (or ones) that match the selected pattern.

As done in class, you will need to access your list of candlesticks and generate a list of smart candlesticks from it. You will then process that list of smart candlesticks with an appropriate recognizer. For instance, given a candlestick pattern called `patternX`, you would have a recognizer class called `recognizer_patternX` (make sure you follow the naming convention). In class we saw that any pattern recognizer should have a method called `recognize` which takes a list of smart candlesticks as input and outputs a list of integers representing the indices of the locations of the pattern in the list of smart candlesticks. If `N` is the index of one location of the pattern, `N` represents the index of the last candlestick in the pattern.

The way you will mark the location of the pattern is via the `Annotation` class. Chart objects have a property that is the list of associations in the chart. You put associations in the chart

by Adding your associations with the Add(...) method. Note that after adding your associations, you need to have the chart redraw itself as it does not do so automatically after each new association is added.

We saw, in class, that in order to use polymorphism, we need to derive our pattern recognizers from a base class called aRecognizer. That base class should provide the recognize() method mentioned earlier. The recognize() method is to scan the list of smart candlestick and pass the appropriate number of candlesticks (via a list) to the match() function. The match function takes a list of smart candlesticks and determines if the candlesticks in the list match the pattern that it is meant to recognize. For example, the match method of a recognizer_patternX returns true when the candlesticks in the list passed to it form a pattern. Otherwise the method returns false.

For polymorphism to work, you will want to use the virtual function designation for the match function (look it up).

Once the call to recognize(...) returns a list of integers, you can now place an Annotation at each of the appropriate candlestick.

You will want to add a CheckBox in your form to signify that you want to clear the annotations before displaying the new set. For instance, suppose you want to show all Marubozus. They will be displayed on the chart. Then, you want to display all the engulfing patterns. Perhaps you want to see how often those patterns occur together. In that case, you want to keep the old annotations (Marubozus) when you add the new annotations (engulfing patterns). However, later, when you decide to look for some other pattern, you might want to clear the annotations before displaying the new set. If the CheckBox (label it, "Keep Annotations") is Checked, then we do not clear the annotations. But if the CheckBox is not checked, then we clear the annotations before displaying the new ones.

Requirements

As for project 2:

- **Additional:** Provide recognizers to recognize the following patterns: Marubozu, Bullish Marubozu, Bearish Marubozu, Engulfing Pattern, Bullish Engulfing Pattern and Bearish Engulfing Pattern. You lose 10 points for each recognizer not implemented correctly.

- **Pattern Selector:** Provide a ComboBox for the selection of patterns to be recognized. IMPORTANT! The ComboBox MUST be loaded at RUNTIME. If you preload it (at design time) you will lose 30 points if not implemented as specified.
- **Annotations:** You MUST use the Annotation class to mark the location of the patterns you find (lose 50 points if not implemented)
- **Clearing Annotations:** Provide the check box that determines whether you clear the annotations or accumulate them. (lose 20 points if not implemented)

And, as for project 2:

- **Compilation:** Your code MUST compile without errors. If your code doesn't compile, your project will receive a grade of 0.
- **Running:** Obviously, your code must run. A project that doesn't run will receive a grade of 0.
- **Easy Start:** The loading button must have TabIndex 0 so that it has the focus when the application starts. Similarly, the starting and ending date should be January 28, 2021 to February 28, 2021. The OpenFileDialog should also be preset to open the "ABBV-daily.csv" file. When your application is started, the main form must immediately show the ABBV_Daily.csv stock between the dates above, before the user selects any other stocks.
- **Naming Convention:** Make sure you name your controls in the way we specified it in class. You are also to use Camel case, For example the button to load a stock should not be called Button1 (as Visual Studio would) but as Button_SomeMeaningfulName (e.g. Button_LoadStock)
- **File naming:** If you are creating new classes, make sure that the name of the file associated with the new class matches the name of the class. This makes it easy to find where a class is defined/implemented.
- **Comments:** Make sure your code is well commented. Functions headers should be preceded with a "///" comment so that the function's usage is automatically displayed. Inside the function implementation should have "///" type comments inside the function to explain the working of the function.

Application Features

1. **Initial Display:** AGAIN, when your application is run, the main form should show the ABBV_Daily.csv stock even before the user selects any other stocks. Think how the standard application starts by using the default constructor of the main form. That is why in project #1, the main form was initially empty until the user selected a stock. Now, the application in project #2 should start with a display of the ABBV_Daily.csv stock with the initial start and end dates specified in the “Easy Start” paragraph. After that, the user can click the load button and select multiple stocks from the list of available CSV files from the *Stock Data* folder. Users should be able to select any number of files representing different stocks.
2. **Stock Selection:** : The application must display a list of available CSV files from the *Stock Data* folder. Users should be able to select any number of files representing different stocks.
3. **Data Display:** Upon selection, the application should read the chosen CSV files and display the data in the Chart control of each individual form.
4. **Date Range Input:** Users must be able to specify a start date and end date to view only the stock data within that range.
5. **Update Functionality:** An *Update* button must be provided. When pressed, the Chart should be refreshed to show data according to the selected date range.
6. **Date Range Modification:** Users should have the option to change the date range and update the display as often as needed in any of the forms.
7. **Period Specification:** The user must be able to display all the stock files or just the daily, just the weekly, monthly or yearly periods.

Technical Specifications

- **Framework:** NET Framework or .NET Core
- **UI Framework:** Windows Forms
- **Controls:** utilize the controls we used in class.
- **Data Handling:** Use appropriate objects from the .NET framework libraries to read and parse CSV files. DO NOT USE outside classes like CSVHelper. You are to write your own data loading class (as we did in class).

Submission Instructions

- Submit the complete Visual Studio solution by CLEANING (Clean in the menu) and then ZIPPING your solution folder (NOT the project folder which is inside the

solution folder). Do not include the “Stock Data” file. That file should be in the parent folder of your solution folder.

- A rubric will be provided shortly.
- I have tried to keep instructions simple to minimize confusion. But feel free to ask questions if you have any.
- You can use the suggestions your instructor makes in class, or do your own implementation, or even use AI to help you design your project. HOWEVER, you want to make sure you fully understand the code you submit. Your level of comprehension of your project will be assessed via a quiz, as was done after project 1.
- You are STRICTLY FORBIDDEN from getting your project from GitHub and submitting it as your own. That would be considered CHEATING and would subject your project to an immediate 0 and your overall grade would be an F,
- Make SURE you understand the specifications. If you need ANY CLARIFICATIONS, ask for them in class!!!

Final instruction regarding the use of AI. For project 3, DO NOT USE code generated from an AI source. Instead make sure you use the code the created in class and MAKE ABSOLUTELY SURE THAT YOU UNDERSTAND IT!!! You have been told that the use of AI is permitted. However, you need to make sure that you TRULY understand the code provided to you. In order to simplify things, you are not to use the code generated by AI to control the Chart control. The only way you are allowed to control the Chart is via the IDE by setting some of its properties (ChartArea, Series, etc....) as we did in class!!! The only allowed approach for displaying the stock data in the chart is by setting the Chart's DataSource and calling its DataBind() method! The last 2 requirements are implemented via 2 lines of code!!!

This can be done in the properties editor in the IDE. So, again, setting a single property will perform the normalization for you!!! You no longer need to write the normalization code (though it is a simple method).

Again, make sure that you understand the code because the quiz will consist in modifying your final project and submitting your updated project 3. If you simply use AI code, you will most likely not be able to make the modifications required during the quiz!